Proiectarea Algoritmilor

Curs 9 – Drumuri de cost minim multi-punct multi-punct



Bibliografie

- [1] R. Sedgewick, K. Wayne - Algorithms and Data Structures Fall 2007 – Curs Princeton - http://www.cs.princeton.edu/~rs/AlgsDS07/
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*
- [3] Giumale – Introducere în Analiza Algoritmilor cap. 5.4, 5.4.2

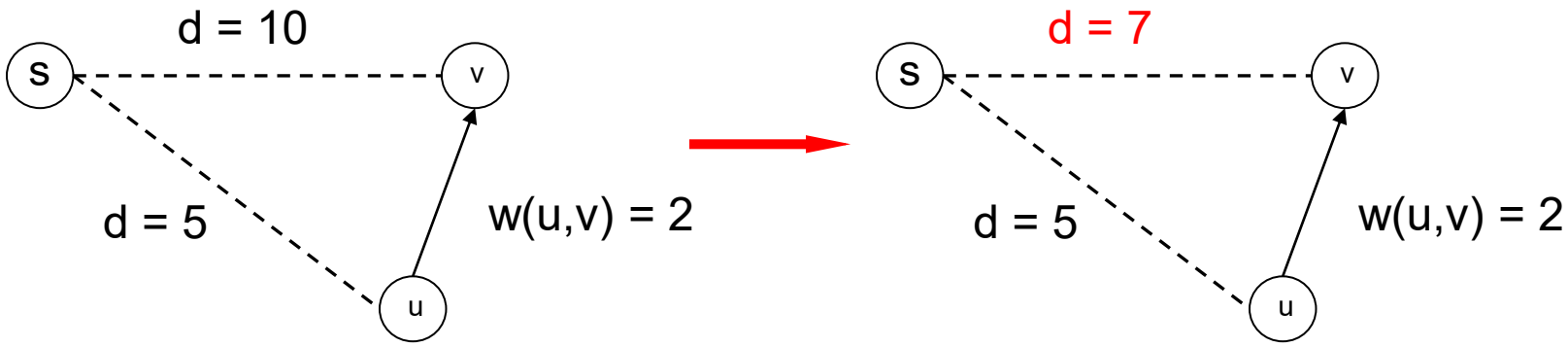
Obiective

- “Descoperirea” algoritmilor de identificare a drumurilor de cost minim.
- Discutarea corectitudinii acestor algoritmi.



Relaxarea arcelor - Reminder

- Relaxarea arcelor:
  - **Dacă** d[v] > d[u] + w(u,v), **atunci**
    - d[v] = d[u] + w(u,v);
    - p[v] = u



Floyd-Warshall (Roy-Floyd)

- Algoritm prin care se calculează distanțele minime între oricare 2 noduri dintr-un graf (drumuri optime multipunct-multipunct).
- Exemplu clasic de programare dinamică.
- Idee: la pasul k se calculează cel mai bun cost între u și v folosind cel mai bun cost u..k și cel mai bun cost k..v calculat până în momentul respectiv.
- Rezultatele furnizate sunt corecte dacă algoritmul se aplică pe grafuri ce nu conțin cicluri de cost negativ.



Notații

- G = (V,E); V = {1, 2, …, n};
- w : V x V → $\mathfrak{R}$; w(i, i) = 0; w(i, j) = ∞ dacă (i,j) ∉ E;
- $d^k(i,j)$ = costul drumului i..j construit astfel încât drumul trece doar prin noduri din mulțimea {1, 2, .., k};
- δ(i,j) = costul drumului optim i..j; δ(i,j) = ∞ dacă ∄ i..j;
- $\delta^k(i,j)$ = costul drumului optim i..j ce trece doar prin noduri din mulțimea {1, 2, .., k}; $\delta^k(i,j)$ = ∞ dacă ∄ un astfel de drum i..j;
- $p^k(i,j)$ = predecesorul lui j pe drumul i..j ce trece doar prin noduri din mulțimea {1, 2, .., k}.

Teorema Floyd - Warshall

- Teoremă: Fie formulele de mai jos pentru calculul valorii $d^k(i,j)$, $0 \le k \le n$:
  - $d^0(i,j) = w(i,j)$;
  - $d^k(i,j) = \min\{d^{k-1}(i,j), d^{k-1}(i,k) + d^{k-1}(k,j)\}$, pentru $0 < k \le n$;

  Atunci $d^n(i,j) = \delta(i,j)$, pentru $\forall\ i, j \in V$

- Dem:
  - Prin inducție după k dem. că $d^k(i,j) = \delta^k(i,j)$. (next slide)
  - Pt. k = n, i..j trece prin $\forall\ v \in V$ si avem $d^k(i,j) \le d^{k-1}(i,j)$, $\forall\ k = 1,n \rightarrow d^n(i,j) \le d^{k-1}(i,j)$, $\forall\ k = 1,n$
  - Din $d^k(i,j) = \delta^k(i,j) \rightarrow d^n(i,j) = \delta^n(i,j) \le d^{k-1}(i,j) = \delta^{k-1}(i,j)$, $\forall\ k = 1,n \rightarrow d^n(i,j) = \delta^n(i,j) = \delta(i,j)$

Demonstrație teorema Floyd - Warshall

$d^k(i,j) = \min\{d^{k-1}(i,j), d^{k-1}(i,k) + d^{k-1}(k,j)\}$

- $k = 0$: 0 noduri intermediare → i..j = (i,j), la fel ca inițializarea $d^0(i,j) = w(i,j)$;
- $0 < k \leq n$: $d^{k-1}(i,j) = \delta^{k-1}(i,j) \rightarrow d^k(i,j) = \delta^k(i,j)$
- a) $k \notin$ drumului optim i..j: drumul optim nu se modifică

  $(\delta^{k-1}(i,j) = \delta^k(i,j) \leq \delta^{k-1}(i,k) + \delta^{k-1}(k,j))$
- $d^k(i,j) = \min\{d^{k-1}(i,j), d^{k-1}(i,k) + d^{k-1}(k,j)\}$

  $d^k(i,j) = \min\{\delta^{k-1}(i,j), \delta^{k-1}(i,k) + \delta^{k-1}(k,j)\} = \delta^{k-1}(i,j) = \delta^k(i,j)$
- b) $k \in$ drumului optim i..j: i..j se descompune în i..k și k..j optime $(\delta^{k-1}(i,k) = d^{k-1}(i,k)$ și $\delta^{k-1}(k,j) = d^{k-1}(k,j))$ și

  $\delta^k(i,j) = \delta^{k-1}(i,k) + \delta^{k-1}(k,j)$.
- i..j optim → $\delta^k(i,j) \leq \delta^{k-1}(i,j)$
- $d^k(i,j) = \min\{d^{k-1}(i,j), d^{k-1}(i,k) + d^{k-1}(k,j)\}$
- $d^k(i,j) = \min\{\delta^{k-1}(i,j), \delta^{k-1}(i,k) + \delta^{k-1}(k,j)\} = \delta^{k-1}(i,k) + \delta^{k-1}(k,j) = \delta^k(i,j)$

Algoritm Floyd-Warshall

- **Floyd-Warshall(G)**
- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n // inițializări
    - $d^0(i,j) = w(i,j)$
    - **Dacă** ($w(i,j) == \infty$)
      - $p^0(i,j) = null;$
    - **Altfel** $p^0(i,j) = i;$
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - **Dacă** ($d^{k-1}(i,j) > d^{k-1}(i,k) + d^{k-1}(k,j)$) // determinăm minimul
        - $d^k(i,j) = d^{k-1}(i,k) + d^{k-1}(k,j)$
        - $p^k(i,j) = p^{k-1}(k,j);$ // și actualizăm părintele
      - **Altfel**
        - $d^k(i,j) = d^{k-1}(i,j)$
        - $p^k(i,j) = p^{k-1}(i,j);$

Complexitate?

$O(V^3)$

Complexitate spațială?

$O(V^3)$

Observație

- Putem folosi o singură matrice în loc de n?
- Problemă: în pasul k, pentru k < i și k < j, d(i,k) și d(k,j) folosite la calculul d(i,j) sunt $d^k(k,j)$ și $d^k(i,k)$ în loc de $d^{k-1}(k,j)$ și $d^{k-1}(i,k)$. Dacă dem. că $d^k(k,j)= d^{k-1}(k,j)$ și $d^k(i,k)=d^{k-1}(i,k)$, atunci putem folosi o singură matrice.
- Dar: $d^k(i,j) = \min\{d^{k-1}(i,j), d^{k-1}(i,k) + d^{k-1}(k,j)\}$
  - $d^k(k,j) = \min\{d^{k-1}(k,j), d^{k-1}(k,k) + d^{k-1}(k,j)\} = d^{k-1}(k,j)$
  - $d^k(i,k) = \min\{d^{k-1}(i,k), d^{k-1}(i,k) + d^{k-1}(k,k)\} = d^{k-1}(i,k)$
- → Algoritm modificat pentru a folosi o singura matrice → complexitate spațială: $O(n^2)$.

Algoritm Floyd-Warshall

Floyd-Warshall2(G)

- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n // inițializări
    - d(i,j) = w(i,j)
    - **Dacă** (w(i,j) == ∞)
      - p(i,j) = null;
    - **Altfel** p(i,j) = i;
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - **Dacă** (d(i,j) > d(i,k) + d(k,j)) // determinăm minimul
        - d(i,j) = d(i,k) + d(k,j)
        - p(i,j) = p(k,j); // și actualizăm părintele

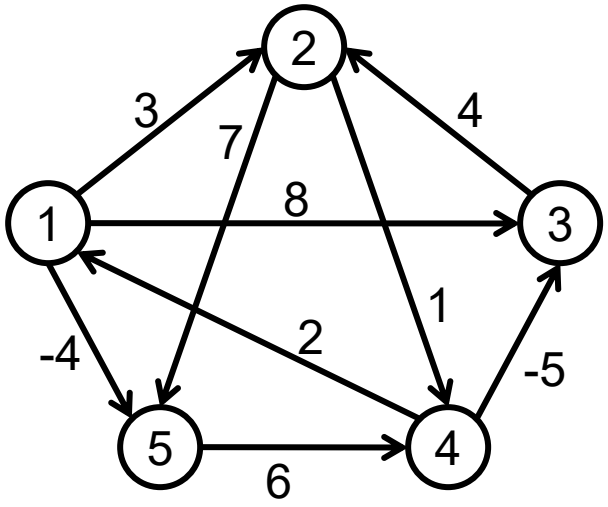
Complexitate?

$O(V^3)$

Complexitate spațială?

$O(V^2)$

Exemplu (I)

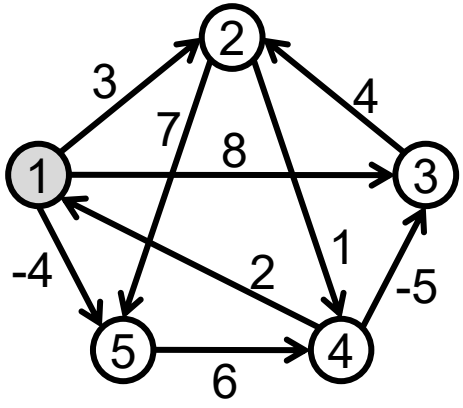


$$D = \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 1 & nil & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & nil & nil \\ 4 & nil & 4 & nil & nil \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

Exemplu (II)

$$D = \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$



$$D = \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

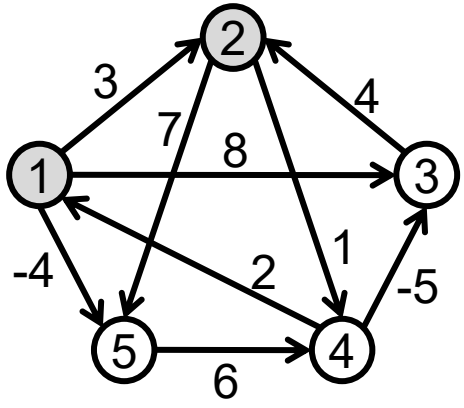
$$p = \begin{bmatrix} nil & 1 & 1 & nil & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & nil & nil \\ 4 & nil & 4 & nil & nil \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 1 & nil & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & nil & nil \\ 4 & 1 & 4 & nil & 1 \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

Exemplu (III)

$$D = \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

$$D = \begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$



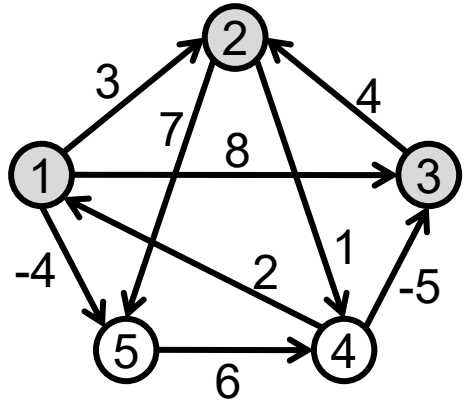
$$p = \begin{bmatrix} nil & 1 & 1 & nil & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & nil & nil \\ 4 & 1 & 4 & nil & 1 \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 1 & 2 & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & 2 & 2 \\ 4 & 1 & 4 & nil & 1 \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

Exemplu (IV)

$$D = \begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

$$D = \begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$



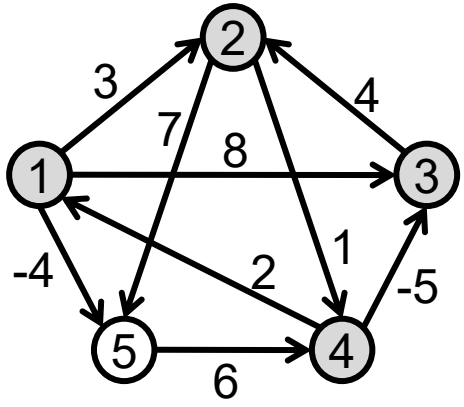
$$p = \begin{bmatrix} nil & 1 & 1 & 2 & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & 2 & 2 \\ 4 & 1 & 4 & nil & 1 \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 1 & 2 & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & 2 & 2 \\ 4 & 3 & 4 & nil & 1 \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

Exemplu (V)

$$D = \begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

$$D = \begin{bmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix}$$

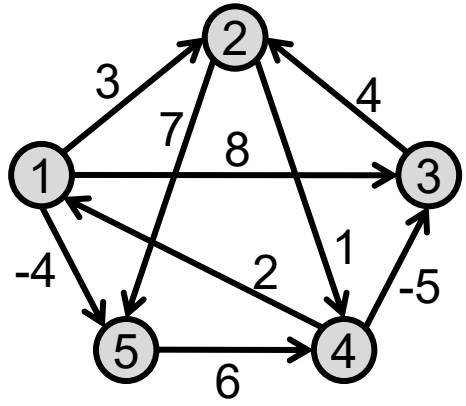


$$p = \begin{bmatrix} nil & 1 & 1 & 2 & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & 2 & 2 \\ 4 & 3 & 4 & nil & 1 \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 4 & 2 & 1 \\ 4 & nil & 4 & 2 & 1 \\ 4 & 3 & nil & 2 & 1 \\ 4 & 3 & 4 & nil & 1 \\ 4 & 3 & 4 & 5 & nil \end{bmatrix}$$

Exemplu (VI)

$$D = \begin{bmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix}$$



$$D = \begin{bmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 4 & 2 & 1 \\ 4 & nil & 4 & 2 & 1 \\ 4 & 3 & nil & 2 & 1 \\ 4 & 3 & 4 & nil & 1 \\ 4 & 3 & 4 & 5 & nil \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 3 & 4 & 5 & 1 \\ 4 & nil & 4 & 2 & 1 \\ 4 & 3 & nil & 2 & 1 \\ 4 & 3 & 4 & nil & 1 \\ 4 & 3 & 4 & 5 & nil \end{bmatrix}$$

Închiderea tranzitivă (I)

- Fie G = (V,E). Închiderea tranzitivă a lui E e un G* = (V,E*), unde

$$E^*(i,j)=\begin{cases}1, \text{ dacă } \exists\ i..j\\ 0, \text{ dacă } \nexists\ i..j\end{cases}$$

- Poate fi determinată prin modificarea algoritmului Floyd-Warshall:
  - min $\Longrightarrow$ operatorul boolean sau ($\vee$)
  - + $\Longrightarrow$ operatorul boolean și ($\wedge$)



Închiderea tranzitivă (II)

Închidere_tranzitivă(G)

- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n
    - $E^*(i,j) = (((i,j) \in E) \vee (i = j))$ // inițializări
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - $E^*(i,j) = E^*(i,j) \vee (E^*(i,k) \wedge E^*(k,j))$

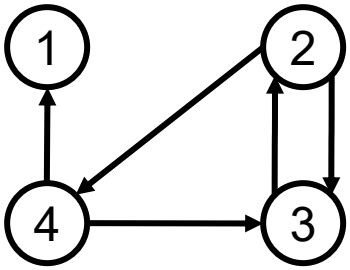
Complexitate? $O(V^3)$

Complexitate spațială? $O(V^2)$

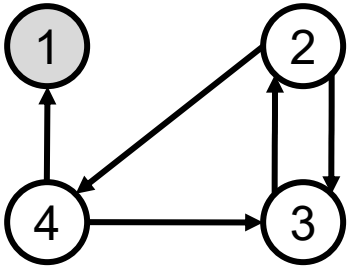
Exemplu (I)

Închidere_tranzitivă(G)

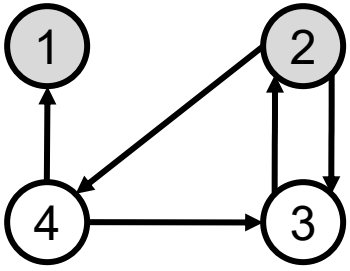
- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n
    - E* (i,j) = (i,j) ∈ E ∨ i = j
      // inițializări
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - E* (i,j) = E* (i,j) ∨ (E* (i,k) ∧ E* (k,j))



$$T^{(0)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$



$$T^{(1)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

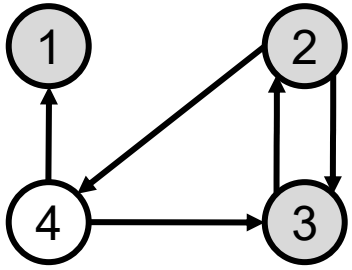


$$T^{(2)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

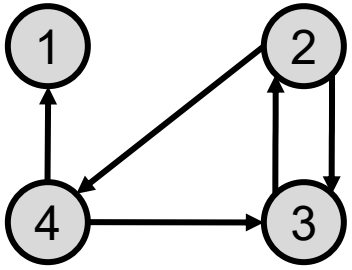
Exemplu (II)

Închidere_tranzitivă(G)

- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n
    - E* (i,j) = (i,j) ∈ E ∨ i = j
      // inițializări
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - E* (i,j) = E* (i,j) ∨ (E* (i,k) ∧ E* (k,j))



$$T^{(3)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

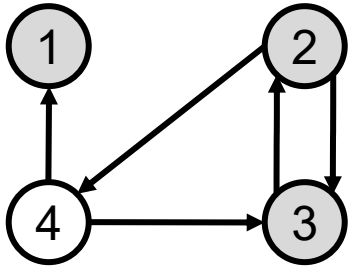


$$T^{(4)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

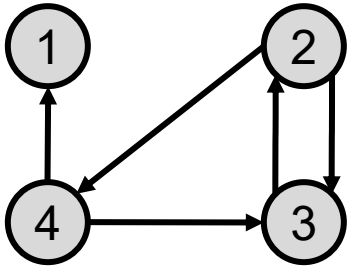
Exemplu (II)

Închidere_tranzitivă(G)

- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n
    - E* (i,j) = (i,j) ∈ E ∨ i = j
      // inițializări
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - E* (i,j) = E* (i,j) ∨ (E* (i,k) ∧ E* (k,j))



$$T^{(3)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$



$$T^{(4)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Algoritmul lui Johnson

- Pentru grafuri rare.
- Folosește liste de adiacență.
- Bazat pe Dijkstra și Bellman-Ford.
- Complexitate: $O(V^2 \log V + VE)$
  - Mai bună decât Floyd-Warshall pentru grafuri rare.

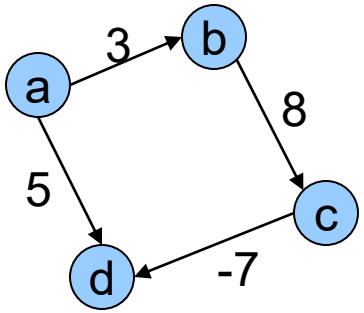


Idee algoritm Johnson

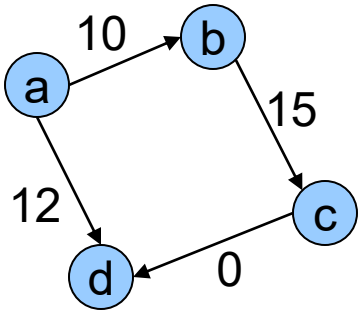
- Dacă graful are numai arce pozitive:
  - se aplică Dijkstra pentru fiecare nod $\Longrightarrow$ cost $V^2 \log V + VE$.
- Altfel se calculează costuri pozitive pentru fiecare arc menținând proprietățile:
  - $w_1(u,v) \geq 0, \forall (u,v) \in E$;
  - p este drum minim utilizând w $\Leftrightarrow$ p este drum minim utilizând w_1.

Construcție w_1 (I)

- Idee 1: identificare arcul cu cel mai mic cost – c; adunare la costul fiecărui arc valoarea c;







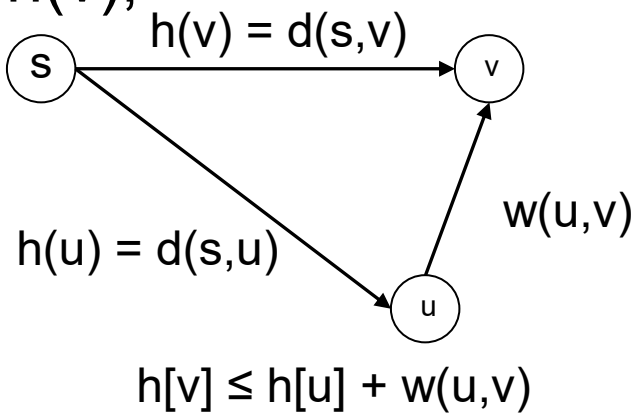
cost(a..b..d) < cost(a,d)

cost(a..b..d) > cost(a,d)

Nu funcționează!!!!

Construcție w_1 (II)

- Idee 2: $w_1(u,v) = w(u,v) + h(u) - h(v)$;
- unde h : V → $\mathfrak{R}$;
- se adaugă un nod s;
- se unește s cu toate nodurile grafului prin arce de cost 0;
- se aplică BF pe acest graf => h(v) = δ(s,v);
- → $w_1(u,v) = w(u,v) + h(u) - h(v)$.



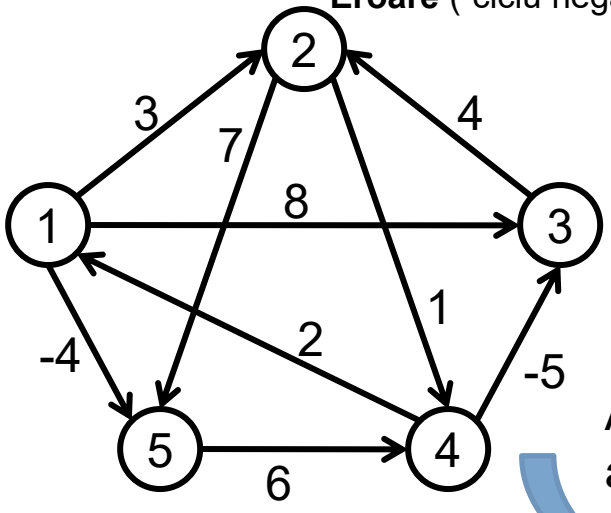
Algoritm Johnson

- Johnson(G)
  - // Construim G' = (V',E');
  - V' = V $\cup$ {s}; // adăugăm nodul s
  - E' = E $\cup$ (s,u), $\forall u \in V$; w(s,u) = 0; // și îl legăm de toate nodurile
  - **Dacă** BF(G',s) e fals // aplic BF pe G'
    - **Eroare** "ciclu negativ"
  - **Altfel**
    - **Pentru fiecare** v $\in$ V
      - h(v) = δ(s,v); // calculat prin BF
    - **Pentru fiecare** (u,v) $\in$ E
      - w_1(u,v) = w(u,v) + h(u) - h(v) // calculez noile costuri pozitive
    - **Pentru fiecare** (u $\in$ V)
      - Dijkstra(G,w_1,u) // aplic Dijkstra pentru fiecare nod
      - **Pentru fiecare** (v $\in$ V)
        - d(u,v) = δ_1(u,v) + h(v) - h(u) // calculez costurile pe graful inițial

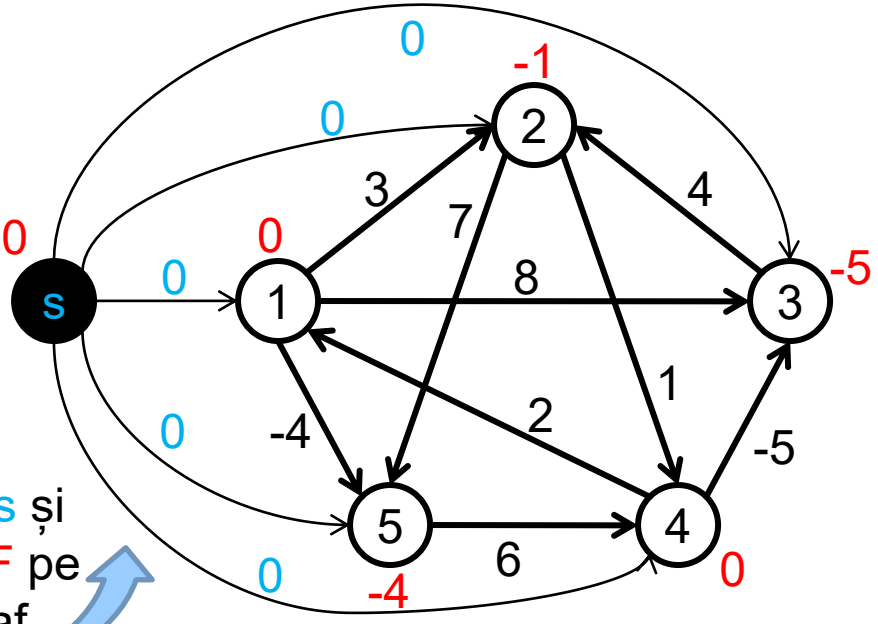


Exemplu (I)

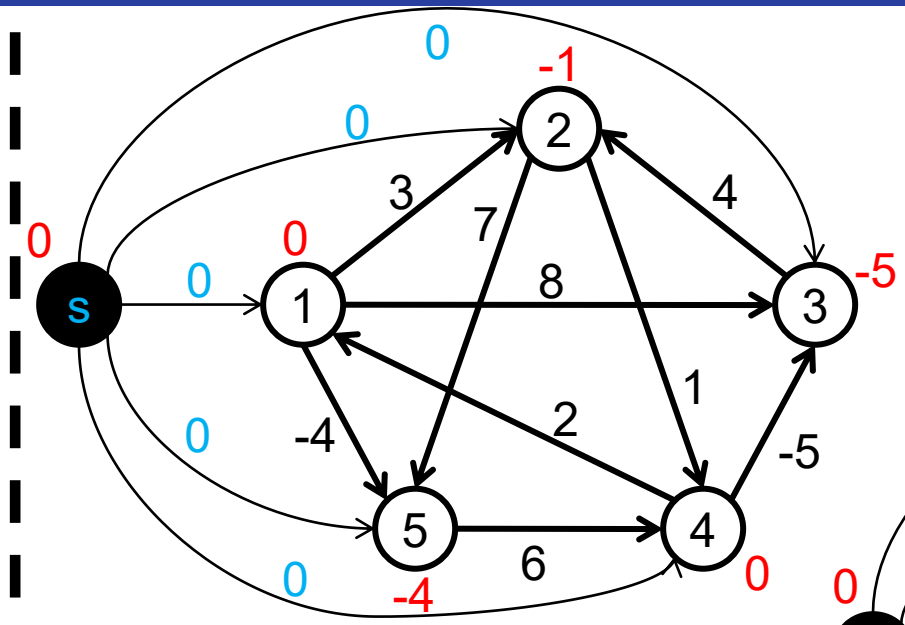
```
BellmanFord(G,s)   // G=(V,E),s=sursa
    Pentru fiecare v din V // inițializări
        d[v] = ∞; p[v] = null;
    d[s] = 0; // actualizare distanță de la s la s
    Pentru i de la 1 la |V| -1 // se construiesc drumuri optime de dimensiune i
        Pentru fiecare (u,v) din E  // pentru arcele ce pleacă de la nodurile deja considerate
            Dacă d[v] > d[u] + w(u,v) atunci // se relaxează arcele corespunzătoare
                d[v] = d[u] + w(u,v);  p[v] = u;
    Pentru fiecare (u,v) din E
        Dacă d[v] > d[u] + w(u,v) atunci
            Eroare ("ciclu negativ");
```



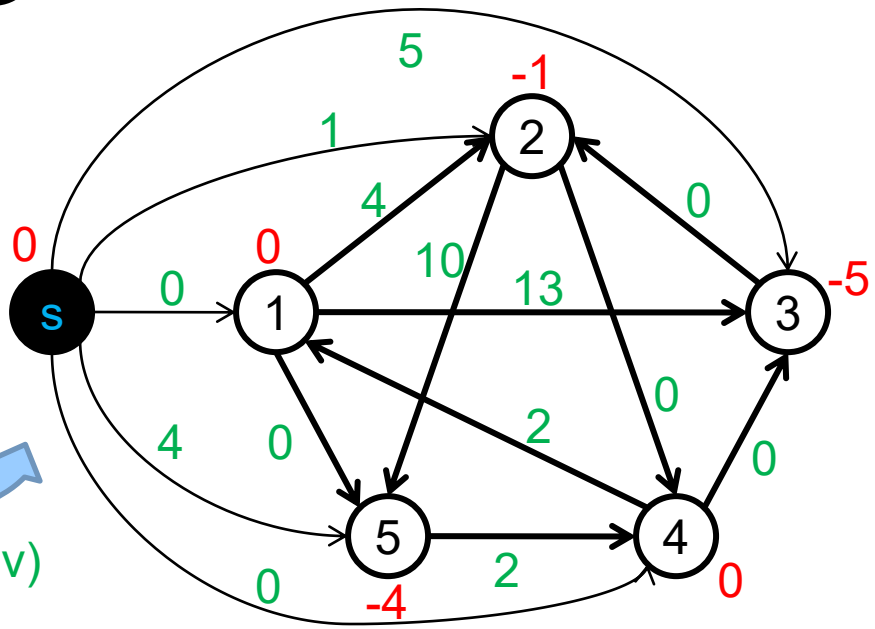
Adaug s și aplic BF pe noul graf.



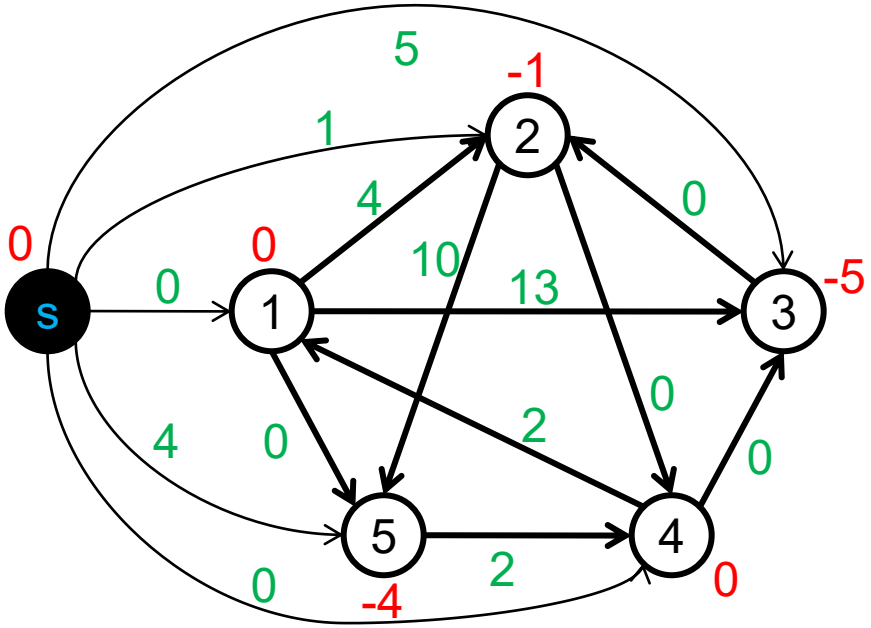
Exemplu (II)

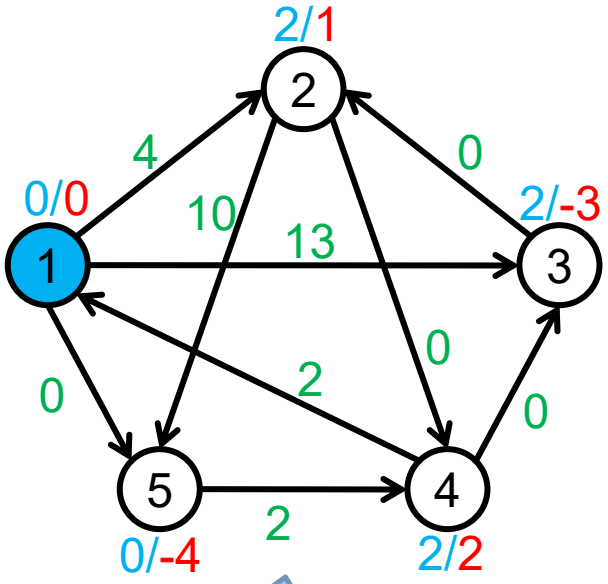


$w_1(u,v) = w(u,v) + h(u) - h(v)$



Exemplu (III)





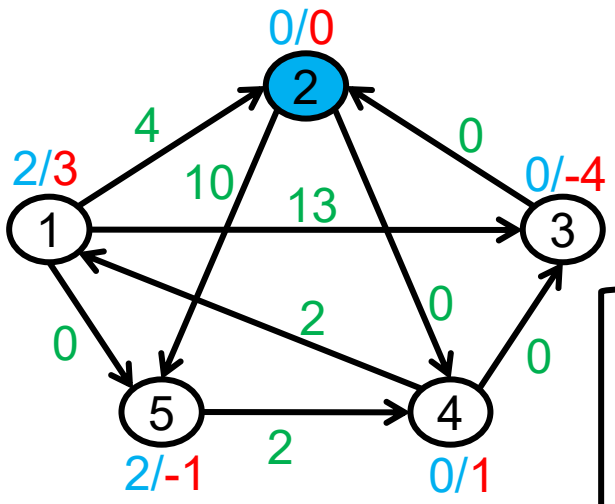
Eliminăm s

Aplicăm Dijkstra din fiecare nod ($\delta_1(u,v)$).

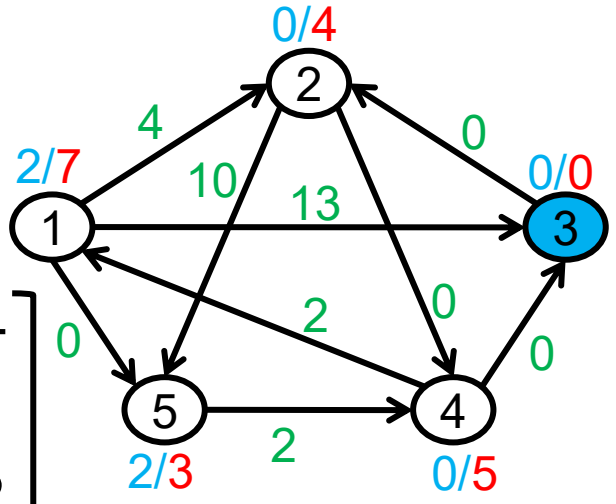
Refacem distanțele:

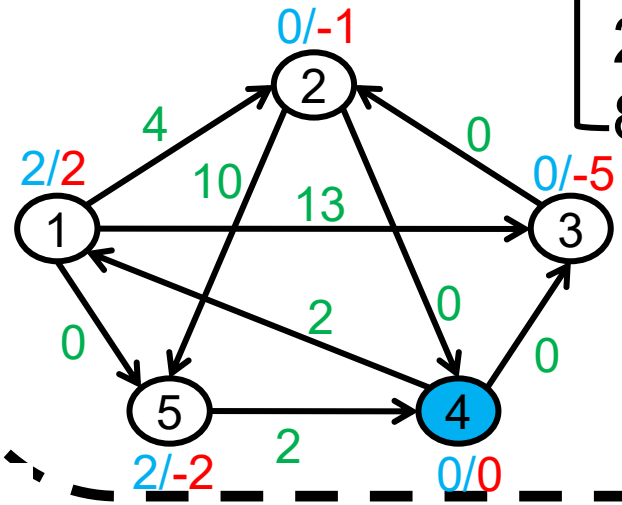
$$d(u,v) = \delta_1(u,v) + h(v) - h(u)$$

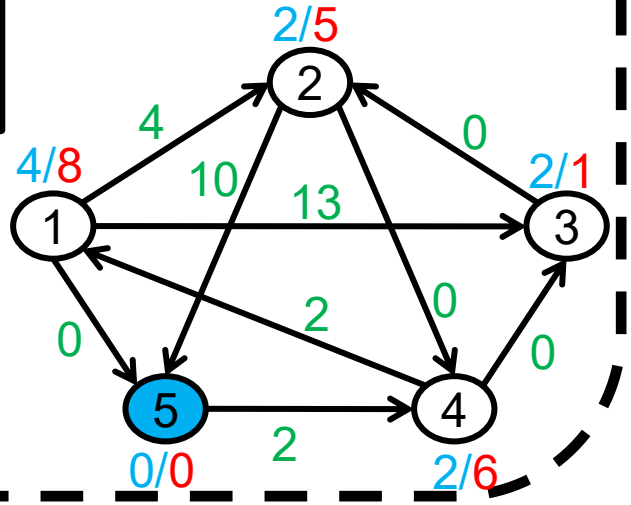
Exemplu (IV)



$$\begin{bmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix}$$







Concluzii Floyd-Warshall & Johnson

- Algoritmi ce găsesc drumurile minime între oricare 2 noduri din graf.
- Furnizează rezultate corecte pentru grafuri cu arce ce au costuri negative (dar care nu au cicluri de cost negativ).
- Floyd-Warshall e optim pentru grafuri dese.
- Johnson e mai bun pentru grafuri rare.



ÎNTREBĂRI?



Bibliografie curs 10

[1] http://monalisa.cern.ch/monalisa__Service_Applications__Monitoring_VRVS.html

[2] https://cdn.images.express.co.uk/img/dynamic/78/590x/secondary/Nuclear-war-USA-899096.jpg

[3] Giumale – Introducere in Analiza Algoritmilor cap. 5.5

[4] R. Sedgewick, K Wayne – curs de algoritmi Princeton 2007
https://sedgewick.io/courses/algorithms/ 1. Union-Find si 14. Minimum Spanning Trees

[5] https://www.pinterest.co.uk/pin/516928863449789442/

[6] Cormen – Introducere în Algoritmi cap. Arbori de acoperire minimi (24)

[7] Tarjan, R. & Van Leeuwen, J. (1984). Worst-Case Analysis of Set Union Algorithms. Journal of the Assoaatton for Computmg Machinery, Vot. 31, No. 2, pp 245--281
https://dl.acm.org/doi/pdf/10.1145/62.2160